

A_Politie_artikelen_filteren

March 12, 2025

```
[321]: import pandas as pd
from docx import Document
from tqdm import tqdm; tqdm.pandas()
import os
import re
import locale; locale.setlocale(locale.LC_TIME, 'nl_NL.UTF-8')
from ast import literal_eval
from nltk.tokenize import sent_tokenize
import nltk
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn import metrics
import krippendorff
import json
```

0.1 Politie woorden detecteren

```
[ ]: df = pd.read_csv("df.csv")
```

```
[363]: #Deze Python-code zoekt naar woorden die te maken hebben met de politie, zoals
      ↪ politie, recherche, opsporing en agent, in een stuk tekst en vervangt ze
      ↪ door het woord "POLICE".

      #Stap voor stap uitleg:

      #De code bevat een functie genaamd detect_police, die één stuk tekst (x) als
      ↪ invoer krijgt.
      #Er is een zoekpatroon (pattern1) dat specifiek kijkt naar de woorden politie,
      ↪ recherche, opsporing en agent, maar alleen als ze op zichzelf staan (dus
      ↪ niet als onderdeel van een langer woord).
      #De re.sub-functie vervangt deze woorden, ongeacht hoofdletters of kleine
      ↪ letters, door "POLICE".
      #De aangepaste tekst wordt teruggegeven.

      def detect_police(x):
          pattern1 = r"\b(politie|recherche|opsporing|agent)\b"
```

```

pattern2 = r"(politie[~k].*|recherche.*|opsporing.*|agent.*)"
new_text = re.sub(pattern1, "POLICE", x, flags=re.IGNORECASE)
return new_text

```

[364]: *#Deze code past de eerder besproken detect_police-functie toe op twee kolommen
↳ van een dataset (df):*

*#Kolom "title": Voor elke titel in de dataset wordt gecontroleerd of er woorden
↳ zoals politie, recherche, opsporing of agent in voorkomen. Als dat zo is,
↳ worden ze vervangen door "POLICE".*

*#Kolom "body": Ditzelfde proces gebeurt voor de hoofdtekst van elk bericht of
↳ artikel in de dataset.*

```

df["title"] = df.title.progress_apply(lambda x: detect_police(x))
df["body"] = df.body.progress_apply(lambda x: detect_police(x))

```

```

100%|          | 42475/42475 [00:00<00:00, 187053.83it/s]

```

```

100%|          | 42475/42475 [00:03<00:00, 12707.46it/s]

```

[365]: *#Deze code telt hoe vaak het woord "POLICE" voorkomt in de titel en de
↳ hoofdtekst van elk item in de dataset (df).*

*#Kolom "title_hits": Hier wordt voor elke titel geteld hoe vaak het woord
↳ "POLICE" erin staat en dat aantal wordt in een nieuwe kolom opgeslagen.*

*#Kolom "body_hits": Ditzelfde gebeurt voor de hoofdtekst van elk item in de
↳ dataset.*

*#Op deze manier wordt bijgehouden hoe vaak termen die met politie te maken
↳ hebben in de tekst zijn vervangen.*

```

df["title_hits"] = df.title.progress_apply(lambda x: x.count("POLICE"))
df["body_hits"] = df.body.progress_apply(lambda x: x.count("POLICE"))

```

```

100%|          | 42475/42475 [00:00<00:00, 342709.47it/s]

```

```

100%|          | 42475/42475 [00:00<00:00, 219978.99it/s]

```

[367]: *#Deze code filtert de dataset (df) en houdt alleen de rijen over waarin:*

#De titel minstens één keer het woord "POLICE" bevat (df.title_hits >= 1).

#De hoofdtekst meer dan één keer het woord "POLICE" bevat (df.body_hits > 1).

*#Alle andere rijen worden verwijderd. Hierdoor blijven alleen de teksten over
↳ waarin politie-gerelateerde termen vaker voorkomen.*

```

df = df[(df.title_hits >= 1) & (df.body_hits > 1)]

```

[369]: *# Deze code voegt de titel en de hoofdtekst samen in een nieuwe kolom genaamd
↳ "text".*

```
df["text"] = df.progress_apply(lambda row: row["title"] + ". " + row["body"],
                                axis=1)
```

100%| | 1501/1501 [00:00<00:00, 81451.75it/s]

```
[370]: # Deze functie haalt zinnen op waarin het woord "POLICE" voorkomt.
# 1. De tekst wordt opgesplitst in losse zinnen met behulp van sent_tokenize.
# 2. Elke zin wordt gecontroleerd op de aanwezigheid van het woord "POLICE".
# 3. Als een zin "POLICE" bevat, wordt deze toegevoegd aan een lijst.
# 4. De lijst met gevonden zinnen wordt als resultaat teruggegeven.
```

```
def get_police_sentences(x):
    police_sentences = []
    sentences = sent_tokenize(x)
    for s in sentences:
        if "POLICE" in s:
            police_sentences.append(s)
    return police_sentences
```

```
[371]: # Deze code past de functie get_police_sentences toe op de "text"-kolom van de
        dataset.
# De functie haalt de zinnen op die het woord "POLICE" bevatten.
# De gevonden zinnen worden opgeslagen in een nieuwe kolom genaamd
        "police_sentences".
```

```
df["police_sentences"] = df.text.progress_apply(lambda x:
                                                get_police_sentences(x))
```

100%| | 1501/1501 [00:00<00:00, 2922.95it/s]

```
[373]: # Deze code verwijdert dubbele rijen uit de dataset op basis van de opgegeven
        kolommen.
```

```
df.drop_duplicates(subset=["title", "channel"], inplace=True)
df.drop_duplicates(subset=["body", "channel"], inplace=True)
```

0.2 Binnenlands nieuws detecteren

```
[375]: # Deze functie bepaalt of een gegeven waarde betrekking heeft op binnenlands of
        buitenlands nieuws.
```

```
def get_domestic(x):
    if x in ["binnenland", "regio", "landelijk", "nederland"]:
        return True
    elif x in ["buitenland"]:
        return False
    else:
        return None
```

```
[376]: # Deze code past de functie get_domestic toe op de "section"-kolom van de
        ↪ dataset.
        # Voor elke waarde in de "section"-kolom wordt gecontroleerd of het betrekking
        ↪ heeft op binnenlands of buitenlands nieuws.

        df["is_domestic"] = df.section.progress_apply(lambda x: get_domestic(x))
```

```
100%|          | 1213/1213 [00:00<00:00, 190400.46it/s]
```

```
[378]: # Deze code filtert de dataset en houdt alleen de rijen over waarin de
        ↪ "is_domestic"-kolom True of False is.
        # 1. Rijen waar "is_domestic" gelijk is aan True of False worden behouden.
        # 2. Rijen waarin "is_domestic" None is, worden verwijderd.
        # De gefilterde dataset wordt opgeslagen in een nieuwe variabele genaamd
        ↪ "df_section".

        df_section = df[(df.is_domestic == True )| (df.is_domestic == False)]
```

```
[380]: # Deze code traint een model om te voorspellen of een tekst betrekking heeft op
        ↪ binnenlands of buitenlands nieuws.

        X = list(df_section["text"]); y = list(df_section["is_domestic"].astype(int))
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = .20,
        ↪ random_state = 1)
        vectorizer = CountVectorizer(min_df=5, max_df=.5)
        X_train_enc = vectorizer.fit_transform(X_train)
        X_test_enc = vectorizer.transform(X_test)
        nb = MultinomialNB()
        nb.fit(X_train_enc, y_train)
        y_pred = nb.predict(X_test_enc)
        rep_nb=metrics.classification_report(y_test, y_pred); print(rep_nb)
```

	precision	recall	f1-score	support
0	0.94	0.94	0.94	36
1	0.96	0.96	0.96	51
accuracy			0.95	87
macro avg	0.95	0.95	0.95	87
weighted avg	0.95	0.95	0.95	87

```
[381]: # Deze code maakt voorspellingen voor de gehele dataset op basis van de
        ↪ getrainde classifier.
        # De voorspellingen worden opgeslagen in een nieuwe kolom genaamd
        ↪ "domestic_prediction".

        df["domestic_prediction"] = nb.predict(vectorizer.transform(df["text"]))
```

0.3 Verwerken van data

```
[387]: # Deze code filtert de dataset en houdt alleen de rijen over waarin de  
        ↳ voorspelling in de "domestic_prediction"-kolom gelijk is aan 1 (binnenlands  
        ↳ nieuws).
```

```
df = df[df.domestic_prediction == 1]
```

```
[389]: df.to_csv("df2.csv")
```